Artificial Intelligence and Machine Learning (AIML) — Comprehensive Study Notes

Subject Type: *Theory/Programming*
Units Covered: *Unit 1 - Unit 2*
Exam Ready: *Yes*
Course Code: *AI-316T*
University: *USICT*

Table of Contents

1. Unit 1: Artificial Intelligence - Problem Solving
    - 1.1 Introduction to Intelligent Agents
    - 1.2 Problem Formulation
    - 1.3 Uninformed Search Strategies
    - 1.4 Informed Search Strategies (Heuristic Search)
    - 1.5 Constraint Satisfaction Problems
2. Unit 2: Logical Reasoning
    - 2.1 Logical Agents
    - 2.2 Propositional Logic
    - 2.3 First-Order Logic (FOL)
    - 2.4 Inference in First-Order Logic
    - 2.5 Forward Chaining and Backward Chaining
    - 2.6 Unification
    - 2.7 Resolution
3. Unit Summary & Practice Questions

Unit 1: Artificial Intelligence - Problem Solving

1.1 Introduction to Intelligent Agents

What is Artificial Intelligence?

Artificial Intelligence (AI) is the study of **rational agents** and their interactions with environments. AI systems perceive their environment through **sensors** and act upon it through **actuators**. An AI agent can have mental properties such as knowledge, belief, intention, and desire.

Definition of AI

AI can be defined as the branch of computer science that aims to create intelligent machines that can mimic human behavior, think like humans, and perform tasks that typically require human intelligence.

What is an Agent?

An **agent** is anything that perceives its environment through sensors and acts upon that environment through actuators. The agent runs in a continuous cycle of **perceiving, thinking, and acting**.

Types of Agents

Agent Type	Sensors	Actuators	Examples
Human Agent	Eyes, ears, other sensory organs	Hands, legs, vocal tract	Humans
Robotic Agent	Cameras, infrared sensors, microphones	Motors, gears, arms	Robots, drones
Software Agent	Keystrokes, file contents, network packets	Display output, file operations	Chatbots, crawlers

Key Components of an Agent

1. **Sensors**: Devices that detect changes in the environment and send information to other electronic devices. Examples: Cameras, microphones, temperature sensors.
2. **Actuators**: Components that convert energy into motion. They are responsible for moving and controlling a system. Examples: Electric motors, gears, rails.
3. **Effectors**: Devices that affect the environment. Examples: Legs, wheels, arms, fingers, wings, display screens.

Intelligent Agents

An **intelligent agent** is an autonomous entity that acts upon an environment using sensors and actuators to achieve goals. Intelligent agents may learn from the environment to achieve their objectives.

Four Rules for an AI Agent

1. **Perception**: An AI agent must have the ability to perceive the environment
2. **Decision Making**: The observation must be used to make decisions
3. **Action**: Decision should result in an action
4. **Rationality**: The action taken by an AI agent must be a rational action

---

Rational Agent

A **rational agent** is an agent which has clear preference, models uncertainty, and acts in a way to maximize its performance measure with all possible actions. A rational agent performs the "right things."

AI is about creating rational agents for game theory and decision theory applications in various real-world scenarios. In reinforcement learning, rational agents receive positive rewards for optimal actions and negative rewards for wrong actions.

Key Difference: Rationality vs Omniscience

- **Rational Agent**: Acts based on available information and expected outcomes

- **Omniscient Agent**: Knows the actual outcome of its actions (theoretical, not possible in reality)

Measuring Rationality

Rationality is measured by the **performance measure** and is judged based on:

1. **Performance Measure**: Defines the success criterion
2. **Agent's Prior Knowledge**: What the agent knows about the environment
3. **Best Possible Actions**: Actions the agent can perform
4. **Sequence of Percepts**: The history of what the agent has perceived

---

Structure of an AI Agent

The structure of an intelligent agent is a combination of **architecture** and **agent program**.

```
Agent = Architecture + Agent Program
```

Key Terms

1. **Architecture**: The machinery on which an AI agent executes (hardware platform)
2. **Agent Function**: Maps a percept sequence to an action
   - Formula: `f: P* → A` (percepts to actions)
3. **Agent Program**: Implementation of the agent function that executes on the physical architecture

---

PEAS Representation

PEAS is a model used to define an AI agent's properties. It consists of four components:

Component	Description
P - Performance Measure	The objective for the success of an agent's behavior
E - Environment	The setting in which the agent operates
A - Actuators	Devices that execute agent's decisions
S - Sensors	Devices that perceive environmental changes

PEAS Examples

Self-Driving Car:

Component	Elements
Performance	Safety, time, legal drive, comfort
Environment	Roads, other vehicles, road signs, pedestrians
Actuators	Steering, accelerator, brake, signal, horn
Sensors	Camera, GPS, speedometer, odometer, accelerometer, sonar

Medical Diagnosis Agent:

Component	Elements
Performance	Healthy patient, minimized cost
Environment	Patient, hospital, staff, tests, treatments
Actuators	Tests, treatments
Sensors	Keyboard (symptom entry)

Vacuum Cleaner Agent:

Component	Elements
Performance	Cleanness, efficiency, battery life
Environment	Room, table, wood floor, carpet, obstacles
Actuators	Wheels, brushes, vacuum extractor
Sensors	Camera, dirt detection sensor, cliff sensor, bump sensor, infrared wall sensor

Part-Picking Robot:

Component	Elements
Performance	Percentage of parts in correct bins
Environment	Conveyor belt with parts, bins
Actuators	Jointed arms, hand
Sensors	Camera, joint angle sensors

The Turing Test

In 1950, **Alan Turing** introduced a test to check whether a machine can think like a human or not. This test is known as the **Turing Test**.

How the Turing Test Works

The test is based on a party game called "Imitation Game" with three players:

1. **Computer** (Player A)
2. **Human Responder** (Player B)
3. **Interrogator** (Player C) - isolated from other two players

The interrogator asks questions through a keyboard and screen to identify which player is the machine. The test evaluates how closely the computer's responses mimic human responses.

Conditions for Passing

The computer passes the test if the interrogator cannot distinguish between the human and machine responses.

Limitations

- Focuses only on verbal behavior
- Does not test physical appearance
- Does not test perceptual abilities

Chatbots that Attempted the Turing Test

1. **ELIZA**: Created by Joseph Weizenbaum (1966) - first chatterbot
2. **Parry**: Created by Kenneth Colby (1972) - simulated paranoid schizophrenia
3. **Eugene Goostman**: Developed in 2001 - convinced 29% of judges it was human (2012)

Chinese Room Argument

Proposed by **John Searle** in 1980, this argument challenges the validity of the Turing Test. Searle argued that:

- Programming a computer may make it understand language syntactically
- But it will not produce real understanding or consciousness
- Machines like ELIZA manipulate keywords and symbols without genuine comprehension

---

Features Required to Pass Turing Test

Feature	Description
Natural Language Processing (NLP)	Communicate in general human language
**Knowledge Representation	Store and retrieve information during the test
Automated Reasoning	Use stored information to answer questions
Machine Learning	Adapt to new changes and detect patterns
Vision (Total Turing Test)	Recognize objects and actions
Motor Control (Total Turing Test)	Act upon requested objects

1.2 Problem Formulation in AI

What is Problem Solving?

Problem-solving is the process of achieving objectives or resolving particular situations. In AI, it involves investigating potential solutions through reasoning techniques, algorithms, and modeling frameworks.

Problem-Solving in AI

AI problem-solving involves:

- Formulating problems appropriately
- Using search algorithms
- Finding reasonable solutions
- Analyzing root causes

Common AI Problems

Problem	Description
Chess	Game-playing AI
N-Queen Problem	Place N queens on N×N chessboard
Tower of Hanoi	Move disks between pegs
Travelling Salesman Problem	Find shortest route visiting all cities
Water-Jug Problem	Measure specific amount using jugs
Cryptarithmetic	Assign digits to letters

Components of Problem Formulation

Component	Description
Initial State	The starting state from which the agent begins
Actions/Operators	Stage that works with specific classes from the initial state
Transition Model	Description of what each action does
State Space	Set of all possible states
Goal Test	Function that checks if goal is achieved
Path Cost	Numeric cost assigned to each path
Solution	Action sequence from start to goal
Optimal Solution	Solution with lowest path cost

1.3 Uninformed Search Strategies

Overview of Search Algorithms

Search algorithms are universal problem-solving methods in AI. **Problem-solving agents** (goal-based agents) use search strategies to solve specific problems and provide optimal results.

Search Algorithm Terminologies

Term	Definition
Search Space	Set of possible solutions
Start State	Initial state where agent begins
Goal Test	Checks if goal state is achieved
Search Tree	Tree representation of search problem
Actions	Available actions for the agent
Transition Model	Description of action effects
Path Cost	Numeric cost of a path
Solution	Action sequence from start to goal
Optimal Solution	Lowest cost solution

Properties of Search Algorithms

Property	Description
Completeness	Guarantees solution if one exists
Optimality	Guarantees best (lowest cost) solution
Time Complexity	Measure of time to complete
Space Complexity	Maximum storage space required

Types of Search Algorithms

```
Search Algorithms
├── Uninformed Search (Blind Search)
│   ├── Breadth-First Search (BFS)
│   ├── Depth-First Search (DFS)
│   ├── Depth-Limited Search (DLS)
│   ├── Iterative Deepening DFS (IDDFS)
│   ├── Uniform Cost Search (UCS)
│   └── Bidirectional Search
└── Informed Search (Heuristic Search)
    ├── Greedy Best-First Search
    └── A* Search
```

Uninformed/Blind Search

Uninformed search does not contain domain knowledge (closeness, location of goal). It operates in a brute-force manner, examining each node until the goal is found.

Characteristics

- Only includes information about traversing the tree
- No information about search space
- Examines every node until goal is found
- Also called **Blind Search**

---

1. Breadth-First Search (BFS)

BFS searches breadthwise, expanding all successor nodes at the current level before moving to the next level.

Characteristics

- Uses **FIFO queue** data structure
- Explores all nodes at depth d before depth d+1
- Guarantees finding the shallowest goal node

Algorithm

```python
from collections import deque

def bfs(graph, start, goal):
    """
    Breadth-First Search Algorithm

    Args:
        graph: Dictionary representing adjacency list
        start: Starting node
        goal: Goal node to find

    Returns:
        Path from start to goal or None if not found
    """
    # Initialize queue with start node and its path
    queue = deque([(start, [start])])
    visited = set()

    while queue:
        # Remove and process the first node in queue
        node, path = queue.popleft()

        # Check if goal is reached
        if node == goal:
            return path

        # Skip if already visited
        if node in visited:
            continue

        # Mark as visited
        visited.add(node)

        # Add all unvisited neighbors to queue
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))

    return None

# Example usage
graph = {
    'S': ['A', 'B'],
    'A': ['C', 'D'],
    'B': ['E', 'F'],
    'C': ['G'],
    'D': ['G'],
    'E': [],
    'F': ['K'],
    'G': [],
    'K': []
}

result = bfs(graph, 'S', 'K')
print(f"BFS Path: {result}")
# Output: BFS Path: ['S', 'B', 'F', 'K']
```

Complexity Analysis

Complexity	Formula	Description
Time	O(b^d)	b = branching factor, d = depth
Space	O(b^d)	Must store all nodes at each level

Advantages

- Complete (finds solution if exists)
- Optimal for uniform path costs (finds shallowest solution)
- Provides solution if any exists

Disadvantages

- Requires lots of memory
- Slow if solution is far from root

---

2. Depth-First Search (DFS)

DFS is a recursive algorithm that explores as deep as possible before backtracking. It follows each path to its greatest depth before moving to the next path.

Characteristics

- Uses **stack** data structure (can be implemented recursively)
- Explores depth before breadth
- Does not necessarily find the shortest path

Algorithm

```python
def dfs(graph, start, goal, visited=None, path=None):
    """
    Depth-First Search Algorithm

    Args:
        graph: Dictionary representing adjacency list
        start: Starting node
        goal: Goal node to find
        visited: Set of visited nodes
        path: Current path from start

    Returns:
        Path from start to goal or None
    """
    if visited is None:
        visited = set()
    if path is None:
        path = []

    # Add current node to path and visited
    path = path + [start]
    visited.add(start)

    # Check if goal is reached
    if start == goal:
        return path

    # Recursively explore all neighbors
    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            result = dfs(graph, neighbor, goal, visited, path)
            if result:
                return result

    return None

# Example usage
graph = {
    'S': ['A', 'B'],
    'A': ['C', 'D'],
    'B': ['E', 'F'],
    'C': ['G'],
    'D': ['G'],
    'E': [],
    'F': ['K'],
    'G': [],
    'K': []
}

result = dfs(graph, 'S', 'K')
print(f"DFS Path: {result}")
# Output: DFS Path: ['S', 'A', 'C', 'G']
```

Complexity Analysis

Complexity	Formula	Description
Time	O(b^m)	m = maximum depth (can be >> d)
Space	O(bm)	Only stores single path

Advantages

- Requires less memory than BFS
- Faster to reach goal if in right path
- Complete in finite state space

Disadvantages

- May get stuck in infinite loop (without cycle checking)
- Not guaranteed to find solution
- Not optimal (may find longer path)

---

3. Depth-Limited Search (DLS)

DLS is similar to DFS with a predetermined depth limit. It solves the infinite path problem in DFS.

Characteristics

- Sets a maximum depth limit (ℓ)
- Nodes at depth limit have no successors
- Two failure conditions:
  - **Standard Failure**: No solution exists
  - **Cutoff Failure**: No solution within depth limit

Algorithm

```python
def depth_limited_search(graph, start, goal, limit, visited=None):
    """
    Depth-Limited Search Algorithm

    Args:
        graph: Graph representation
        start: Starting node
        goal: Goal node
        limit: Maximum depth to search
        visited: Visited nodes set

    Returns:
        Path if found, 'cutoff' if limit reached, None otherwise
    """
    return dls_recursive(graph, start, goal, limit, visited or set(), 0)

def dls_recursive(graph, node, goal, limit, visited, depth):
    # Check if goal found
    if node == goal:
        return [node]

    # Check if limit reached
    if depth >= limit:
        return 'cutoff'

    visited.add(node)

    # Explore neighbors
    for neighbor in graph.get(node, []):
        if neighbor not in visited:
            result = dls_recursive(graph, neighbor, goal, limit, visited, depth + 1)
            if result == 'cutoff':
                continue  # Continue searching other branches
            if result:  # Found a solution
                return [node] + result

    return None
```

Complexity Analysis

Complexity	Formula
Time	O(b^ℓ)
Space	O(b×ℓ)

---

4. Iterative Deepening Depth-First Search (IDDFS)

IDDFS combines DFS and BFS. It finds the best depth limit by gradually increasing it until the goal is found.

How It Works

1. Perform DFS with depth limit 0
2. If not found, increase limit by 1
3. Repeat until goal is found

Algorithm

```python
def iterative_deepening_dfs(graph, start, goal, max_depth=100):
    """
    Iterative Deepening DFS Algorithm

    Gradually increases depth limit until goal is found

    Args:
        graph: Graph representation
        start: Starting node
        goal: Goal node
        max_depth: Maximum depth to search

    Returns:
        Path if found, None otherwise
    """
    for limit in range(max_depth):
        result = depth_limited_search(graph, start, goal, limit)
        if result and result != 'cutoff':
            return result
    return None
```

Example Traversal

```
Tree: S → A, B → ..., Goal = K

Iteration 1: S
Iteration 2: S, A, B
Iteration 3: S, A, D, E, B, C, F, G
Iteration 4: S, A, D, H, I, E, B, C, F, K (GOAL FOUND!)
```

Complexity Analysis

Complexity	Formula	Description
Time	O(b^d)	Similar to BFS
Space	O(bd)	Similar to DFS

Advantages

- Combines benefits of BFS and DFS
- Memory efficient like DFS
- Complete and optimal (for uniform costs)

Disadvantages

- Repeats work from previous iterations

---

5. Uniform Cost Search (UCS)

UCS is used for traversing weighted trees/graphs. It finds the path with the lowest cumulative cost.

Characteristics

- Expands nodes based on path cost from root
- Uses **priority queue** (minimum cost first)
- Equivalent to BFS if all edge costs are equal

Algorithm

```python
import heapq

def uniform_cost_search(graph, start, goal):
    """
    Uniform Cost Search Algorithm

    Finds path with minimum cumulative cost

    Args:
        graph: Dictionary with node -> [(neighbor, cost), ...]
        start: Starting node
        goal: Goal node

    Returns:
        Path and cost or None
    """
    # Priority queue: (cost, node, path)
    priority_queue = [(0, start, [start])]
    visited = set()

    while priority_queue:
        cost, node, path = heapq.heappop(priority_queue)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            return path, cost

        for neighbor, edge_cost in graph.get(node, []):
            if neighbor not in visited:
                new_cost = cost + edge_cost
                heapq.heappush(priority_queue,
                              (new_cost, neighbor, path + [neighbor]))

    return None

# Example with costs
graph = {
    'S': [('A', 1), ('B', 2)],
    'A': [('C', 1), ('D', 3)],
    'B': [('E', 2), ('F', 1)],
    'C': [('G', 2)],
    'D': [('G', 1)],
    'E': [],
    'F': [('K', 3)],
    'G': [],
    'K': []
}

result = uniform_cost_search(graph, 'S', 'K')
print(f"UCS Path: {result}")
```

Complexity Analysis

Complexity	Formula	Description
Time	O(b^(1 + C*/ε))	C* = optimal cost, ε = minimum cost
Space	O(b^(1 + C*/ε))	Similar to time

Advantages

- Optimal (finds least cost path)
- Complete (if solution exists)

Disadvantages

- May get stuck in infinite loop with zero-cost cycles
- Only concerned with cost, not number of steps

---

6. Bidirectional Search

Bidirectional search runs two simultaneous searches:

- **Forward search**: From initial state
- **Backward search**: From goal state

Stops when the two searches intersect.

Characteristics

- Uses BFS, DFS, or DLS for both directions
- Requires knowing the goal state in advance

Complexity Analysis

Complexity	Formula
Time	O(b^(d/2))
Space	O(b^(d/2))

Advantages

- Faster than unidirectional search
- Requires less memory

Disadvantages

- Difficult to implement
- Requires goal state knowledge

1.4 Informed Search Strategies (Heuristic Search)

What are Heuristics?

A **heuristic** is a technique that solves problems faster than classical methods. Heuristics find approximate solutions when exact methods are impractical.

Definition

Heuristics are strategies derived from past experience with similar problems. They use practical methods and shortcuts to produce solutions that may not be optimal but are sufficient within time constraints.

Why Use Heuristics?

- Required for short-term solutions
- Handle complex situations with limited resources
- Use mental shortcuts based on past experiences

- Not guaranteed to find the best solution, but good solutions in reasonable time

Types of Heuristic Techniques

Type	Description	Examples
Direct Heuristic Search	Blind search with arbitrary ordering	BFS, DFS
Weak Heuristic Search	Domain-specific information	Best-First Search, A*

---

1. Hill Climbing Algorithm

Hill Climbing is a local search algorithm that moves in the direction of increasing elevation/value to find the peak.

Characteristics

- Greedy local search (looks at immediate neighbors only)
- Does not maintain search tree/graph
- Terminates at peak where no neighbor has higher value
- Used when good heuristic is available

State-Space Diagram

```
                    Global Maximum
                         *
                        / \
                       /   \
                      /     \
          Local      /       \        Local
          Maximum   *         \      Maximum
                    \         /
                     \       /
                      \     /
                       \   /
                        \ /
                    Shoulder
                       -
                    Plateau
```

Types of Hill Climbing

Simple Hill Climbing

```python
def simple_hill_climbing(problem):
    """
    Simple Hill Climbing Algorithm

    Evaluates one successor at a time
    """
    current = problem.initial_state()

    while True:
        # Get next state
        next_state = current.best_successor()

        # If no better successor, we're done
        if next_state.value() <= current.value():
            return current

        current = next_state
```

Steepest-Ascent Hill Climbing

```python
def steepest_ascent_hill_climbing(problem):
    """
    Steepest-Ascent Hill Climbing

    Examines all neighbors and selects the best one
    """
    current = problem.initial_state()

    while True:
        # Find best successor
        best_successor = None
        best_value = current.value()

        for successor in current.successors():
            if successor.value() > best_value:
                best_successor = successor
                best_value = successor.value()

        # If no improvement, we're at local maximum
        if best_value <= current.value():
            return current

        current = best_successor
```

Stochastic Hill Climbing

Selects a random neighbor and decides whether to move to it or explore another state.

Problems in Hill Climbing

Problem	Description	Solution
Local Maximum	Peak better than neighbors but not global	Backtracking, maintain promising path list
Plateau	Flat area with same value neighbors	Take big jumps or random states
Ridges	Steep area but cannot reach in single move	Bidirectional search

---

2. Best-First Search (Greedy Search)

Greedy Best-First Search always selects the path that appears best at that moment. It combines DFS and BFS advantages.

Characteristics

- Uses heuristic function h(n)
- Expands node closest to goal
- Formula: f(n) = h(n)
- Implemented using priority queue

Algorithm

```python
def greedy_best_first_search(graph, start, goal, heuristic):
    """
    Greedy Best-First Search Algorithm

    Expands node with lowest heuristic value

    Args:
        graph: Graph representation
        start: Starting node
        goal: Goal node
        heuristic: Dictionary of h(n) values

    Returns:
        Path from start to goal
    """
    # Priority queue: (heuristic_value, node, path)
    priority_queue = [(heuristic[start], start, [start])]
    visited = set()

    while priority_queue:
        _, node, path = heapq.heappop(priority_queue)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            return path

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                heapq.heappush(priority_queue,
                               (heuristic[neighbor], neighbor, path + [neighbor]))

    return None
```

Complexity Analysis

Complexity	Value
Time	O(b^m)
Space	O(b^m)

Advantages

- More efficient than BFS/DFS

- Can switch between BFS and DFS

Disadvantages

- Not optimal
- Can get stuck in loops
- Can behave as unguided DFS

---

3. A* Search Algorithm

A* is the most commonly known form of best-first search. It combines both:

- **g(n)**: Cost to reach node n from start
- **h(n)**: Estimated cost from n to goal

Formula

```
f(n) = g(n) + h(n)
```

Where:

- g(n) = Actual cost from start to n
- h(n) = Heuristic estimate from n to goal
- f(n) = Total estimated cost

Algorithm

```python
def a_star_search(graph, start, goal, heuristic):
    """
    A* Search Algorithm

    Combines g(n) and h(n) for optimal path finding

    Args:
        graph: Dictionary with node -> [(neighbor, cost), ...]
        start: Starting node
        goal: Goal node
        heuristic: Dictionary of h(n) values

    Returns:
        Path and cost or None
    """
    # Priority queue: (f_score, g_score, node, path)
    open_set = [(heuristic[start], 0, start, [start])]
    closed_set = set()

    while open_set:
        f, g, node, path = heapq.heappop(open_set)

        if node in closed_set:
            continue

        closed_set.add(node)

        if node == goal:
            return path, g

        for neighbor, cost in graph.get(node, []):
            if neighbor not in closed_set:
                new_g = g + cost
                new_f = new_g + heuristic.get(neighbor, float('inf'))
                heapq.heappush(open_set,
                              (new_f, new_g, neighbor, path + [neighbor]))

    return None
```

Example Walkthrough

```
Graph with costs and heuristics:

        S(0)
       / \
   A(1)/   \B(2)
     /      \
   C(4)     F(5)
    |\      /|
   D(5)   G(6)
    |     /|
    K(6)--+

Heuristic h(n): S=6, A=4, B=5, C=3, D=4, F=4, G=3, K=0

Step-by-step:
1. Start: {S, f=6}
2. Expand S: {A(f=4), B(f=7)}
3. Expand A: {C(f=4), B(f=7), D(f=8)}
4. Expand C: {D(f=6), G(f=6), B(f=7)}
5. Expand G: {K(f=6)} → GOAL!

Optimal Path: S → A → C → G → K (Cost: 6)
```

Admissible Heuristic

A heuristic h(n) is **admissible** if it never overestimates the cost to reach the goal. It must be optimistic.

Consistent Heuristic

A heuristic is **consistent** if:

```
h(n) ≤ cost(n, n') + h(n')
```

For tree search, admissible heuristic is sufficient. For graph search, both admissible and consistent are needed.

Complexity Analysis

Complexity	Formula
Time	O(b^d)
Space	O(b^d)

Advantages

- Optimal (with admissible heuristic)
- Complete
- Most efficient for finding shortest path

Disadvantages

- Memory intensive (stores all generated nodes)
- Not always produce shortest path (depends on heuristic quality)

1.5 Constraint Satisfaction Problems (CSP)

Definition

A **Constraint Satisfaction Problem** consists of:

- A set of **variables** $\{x_1, x_2, ..., x_n\}$
- Each variable has a **domain** of possible values $\{D_1, D_2, ..., D_n\}$
- A set of **constraints** $\{C_1, C_2, ..., C_m\}$

Formal Definition: CSP is a triple (X, D, C)

Component	Description
X	Set of variables $\{x_1, x_2, ..., x_n\}$
D	Set of domains $\{D_1, D_2, ..., D_n\}$
C	Set of constraints $\{C_1, C_2, ..., C_m\}$

Types of Variables (Domains)

Type	Description	Examples
Finite	Finite number of values	Colors, integers
Infinite	Infinite number of values	Real numbers
Continuous	Continuous range of values	Polynomial coefficients

---

Types of Constraints

Constraint	Description	Example
Unary	Constraint on single variable	A ≠ Red
Binary	Constraint between two variables	A ≠ B
Global	Constraint on multiple variables	AllDifferent(A,B,C)

Solving CSPs

State-Space Approach

```
1. Analyze the problem
2. Derive constraints
3. Find solution from constraints
4. Check if goal state reached
   - If no, make guess and add as new constraint
5. Repeat until solution found
```

Backtracking Search

```python
def backtracking_search(csp):
    """
    Backtracking Search for CSP

    Recursively assigns values to variables
    """
    if is_complete(csp):
        return csp assignment

    var = select_unassigned_variable(csp)

    for value in order_domain_values(csp, var):
        if is_consistent(csp, var, value):
            assign(csp, var, value)
            result = backtracking_search(csp)
            if result:
                return result
            unassign(csp, var, value)

    return None
```

Forward Checking

After assigning a value, remove inconsistent values from neighbors' domains.

Cryptarithmetic Problem Example

```
  SEND
+ MORE
------
 MONEY
```

Variables

S, E, N, D, M, O, R, Y (each = 0-9)

Constraints

1. Each letter = unique digit
2. Leading letters (S, M) ≠ 0
3. Column-wise arithmetic must hold

Solution Process

Column	Constraint	Reasoning
Y	D + E = Y (or Y + 10)	Units digit
C1	N + R = E (or E + 10)	Tens digit
C2	E + O = N (or N + 10)	Hundreds
C3	S + M = O (or O + 10)	Thousands
C4	Carry from C3 = M	Ten-thousands

Unit 2: Logical Reasoning

2.1 Logical Agents

Knowledge-Based Agents

A **knowledge-based agent** uses a knowledge base (KB) to store knowledge and an inference engine to derive new knowledge.

Components

1. **Knowledge Base (KB)**: Set of sentences representing knowledge
2. **Inference Engine**: Derives new conclusions from KB
3. **TELL/ASK Interface**: Adds new knowledge and queries

Architecture

```
┌──────────────────────────────────────┐
│         Knowledge-Based              │
│              Agent                   │
├──────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐  │
│  │  Knowledge   │  │   Inference  │  │
│  │    Base      │  │    Engine    │  │
│  └──────────────┘  └──────────────┘  │
│         ↑                ↓          │
│  ┌──────────────────────────────┐    │
│  │       TELL / ASK             │    │
│  └──────────────────────────────┘    │
└──────────────────────────────────────┘
```

2.2 Propositional Logic

What is Propositional Logic?

Propositional Logic (or Propositional Calculus) is a formal system where:

- **Propositions** are statements that can be true or false
- **Logical connectives** combine propositions
- **Inference rules** derive new true statements

Syntax Elements

Element	Symbol	Example
Propositional Symbols	P, Q, R...	"It is raining"
Negation	¬	¬P = "It is not raining"
Conjunction	∧	P ∧ Q = "It is raining AND cold"
Disjunction	∨	P ∨ Q = "It is raining OR cold"
Implication	→	P → Q = "If it rains, then it's cloudy"
Biconditional	↔	P ↔ Q = "It rains if and only if it's humid"

Truth Tables

Basic Connectives

P	Q	¬P	P ∧ Q	P ∨ Q	P → Q	P ↔ Q
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Logical Equivalence

Two formulas are logically equivalent if they have the same truth value in all interpretations.

Equivalence	Formula
Identity	P ↔ P
Double Negation	¬¬P ↔ P
De Morgan's Laws	¬(P ∧ Q) ↔ (¬P ∨ ¬Q)
	¬(P ∨ Q) ↔ (¬P ∧ ¬Q)
Implication	P → Q ↔ ¬P ∨ Q
Contrapositive	P → Q ↔ ¬Q → ¬P

Rules of Inference

Rule	Name	Pattern
Modus Ponens	Affirming	$P \to Q, P \vdash Q$
Modus Tollens	Denying	$P \to Q, \neg Q \vdash \neg P$
Hypothetical Syllogism	Chain	$P \to Q, Q \to R \vdash P \to R$
Disjunctive Syllogism		$P \vee Q, \neg P \vdash Q$
Addition		$P \vdash P \vee Q$
Simplification		$P \wedge Q \vdash P$

2.3 First-Order Logic (FOL)

Introduction

First-Order Logic (FOL) extends propositional logic by introducing:

- **Quantifiers** (∀, ∃)
- **Predicates** (properties and relations)
- **Variables** (represent objects)
- **Functions** (map objects to objects)

Also known as **First-Order Predicate Calculus** or **Predicate Logic**.

Limitations of Propositional Logic

Propositional Logic	Problem
Cannot express "All humans are mortal"	No variables
Cannot express "John is human" with relation	No predicates
Must enumerate all facts	No quantifiers

FOL Syntax Elements

Element	Example	Description
Constants	KingJohn, 2, UCB	Specific objects
Variables	x, y, a, b	Represent objects
Predicates	Brother(x,y), > (x,3)	Properties/relations
Functions	Father(x), Sqrt(x)	Return objects
Connectives	∧, ∨, ¬, →, ↔	Same as propositional
Quantifiers	∀, ∃	Universal, Existential
Equality	=	Identity relation

Atomic Sentences

An atomic sentence in FOL is a predicate applied to terms:

```
Brother(KingJohn, Richard)
> (Sqrt(x), 3)
Father(Father(John))
```

Quantifiers

Universal Quantifier (∀)

"For all" - states that a property holds for all objects.

```
∀x King(x) → Mortal(x)
"All kings are mortal"

∀x (King(x) ∧ Greedy(x) → Evil(x))
"All king who are greedy are evil"
```

Existential Quantifier (∃)

"There exists" - states that there is at least one object with a property.

```
∃x King(x)
"There exists a king"

∃x (King(x) ∧ Brother(x, John))
"There is a king who is John's brother"
```

Relationship Between Quantifiers

```
∀x P(x) ≡ ¬∃x ¬P(x)
∃x P(x) ≡ ¬∀x ¬P(x)
```

FOL Syntax Grammar

```
Sentence     → AtomicSentence
             | ¬Sentence
             | Sentence ∧ Sentence
             | Sentence ∨ Sentence
             | Sentence → Sentence
             | Sentence ↔ Sentence
             | ∀Variable Sentence
             | ∃Variable Sentence

AtomicSentence → Predicate(Term, ..., Term)
               | Term = Term

Term         → Function(Term, ..., Term)
             | Constant
             | Variable
```

2.4 Inference in First-Order Logic

Types of Inference

1. **Reduction to Propositional Inference**: Convert FOL to propositional logic
2. **Generalized Modus Ponens**: Lifted version of propositional modus ponens
3. **Unification**: Finding substitutions to make expressions identical
4. **Forward Chaining**: Data-driven reasoning
5. **Backward Chaining**: Goal-driven reasoning
6. **Resolution**: Refutation-based theorem proving

Universal and Existential Instantiation

Universal Instantiation (UI)

Any instantiation of a universally quantified sentence can be added:

```
∀x P(x)
---------
P(A)  [where A is any constant]
```

Existential Instantiation (EI)

One instantiation of an existentially quantified sentence:

```
∃x P(x)
---------
P(A)  [where A is a new Skolem constant]
```

Reduction to Propositional Inference

Process:

1. Eliminate existential quantifiers via Skolemization
2. Universal instantiation creates all possible ground sentences
3. Use propositional inference

4. **Problem**: Infinite possibilities with function symbols

2.5 Forward Chaining and Backward Chaining

Forward Chaining

Forward Chaining starts from known facts and applies inference rules to derive new facts until the goal is reached.

Characteristics

- **Data-driven**: Starts with what is known
- **Bottom-up reasoning**: From facts to conclusions
- Used in **deductive databases** and **production systems**

Algorithm

```python
def forward_chaining(KB, goal):
    """
    Forward Chaining Algorithm

    Args:
        KB: Knowledge base with rules
        goal: Goal predicate to prove

    Returns:
        True if goal can be derived, False otherwise
    """
    # Inferred facts
    inferred = set(KB.facts)
    new_facts = set(KB.facts)

    while new_facts:
        new_inferred = set()

        for rule in KB.rules:
            # Check if rule premise can be satisfied
            for fact in new_facts:
                # Try to match premise with fact
                substitution = unify(rule.premise, fact)
                if substitution:
                    # Apply rule to derive conclusion
                    conclusion = apply_substitution(rule.conclusion,
                                                    substitution)
                    if conclusion not in inferred:
                        new_inferred.add(conclusion)
                        print(f"Inferred: {conclusion}")

        new_facts = new_inferred
        inferred.update(new_facts)

        # Check if goal is reached
        if goal in inferred:
            return True

    return False
```

Example

```
KB:
1. King(John)
2. Greedy(John)
3. ∀x (King(x) ∧ Greedy(x) → Evil(x))

Forward Chaining:
Step 1: Start with facts {King(John), Greedy(John)}
Step 2: Apply Modus Ponens to rule 3
Step 3: Infer Evil(John)
Step 4: Goal reached!
```

Backward Chaining

Backward Chaining starts from the goal and works backwards to find supporting facts.

Characteristics

- **Goal-driven**: Starts with what we want to prove
- **Top-down reasoning**: From goals to facts
- Used in **logic programming** (e.g., Prolog)

Algorithm

```python
def backward_chaining(KB, goal, visited=None):
    """
    Backward Chaining Algorithm

    Args:
        KB: Knowledge base
        goal: Goal predicate to prove
        visited: Set of already proved goals (avoid loops)

    Returns:
        True if goal can be proved
    """
    if visited is None:
        visited = set()

    # Goal already proved or in progress
    if goal in visited:
        return False

    visited.add(goal)

    # Check if goal is a known fact
    if goal in KB.facts:
        return True

    # Try each rule that can conclude the goal
    for rule in KB.get_rules_for(goal):
        # Try to prove all premises
        all_proved = True
        for premise in rule.premises:
            if not backward_chaining(KB, premise, visited):
                all_proved = False
                break

        if all_proved:
            return True

    return False
```

Example

```
Goal: Evil(John)

Step 1: Look for rule concluding Evil(x)
        Found: ∀x (King(x) ∧ Greedy(x) → Evil(x))

Step 2: Need to prove King(John) ∧ Greedy(John)
        - King(John) is a fact → TRUE
        - Greedy(John) is a fact → TRUE

Step 3: Goal proved!
```

Comparison: Forward vs Backward Chaining

Aspect	Forward Chaining	Backward Chaining
Direction	Forward (facts → goal)	Backward (goal → facts)
Reasoning	Bottom-up	Top-down
Control	Data-driven	Goal-driven
Use Case	Deductive databases	Expert systems, Prolog
Exploration	All possible conclusions	Only relevant facts
Efficiency	May derive irrelevant facts	More focused

2.6 Unification

What is Unification?

Unification is the process of finding a substitution that makes two logical expressions identical.

Definition

For atomic sentences p and p', find substitution θ such that:

```
SUBST(θ, p) = SUBST(θ, p')
```

Unify Algorithm

```python
def unify(x, y, theta=None):
    """
    Unification Algorithm

    Returns the most general unifier (MGU) for two expressions

    Args:
        x: First expression
        y: Second expression
        theta: Current substitution

    Returns:
        Most general unifier or failure
    """
    if theta is None:
        theta = {}

    # If both expressions equal, return theta
    if x == y:
        return theta

    # If x is a variable
    if is_variable(x):
        return unify_var(x, y, theta)

    # If y is a variable
    if is_variable(y):
        return unify_var(y, x, theta)

    # If both are compound expressions
    if is_compound(x) and is_compound(y):
        if get_predicate(x) != get_predicate(y):
            return failure

        return unify(get_args(x), get_args(y), theta)

    return failure

def unify_var(var, x, theta):
    """Unify a variable with an expression"""
    if var in theta:
        return unify(theta[var], x, theta)

    if occurs_check(var, x):
        return failure

    # Add binding to theta
    theta[var] = x
    return theta
```

Examples

Expression 1	Expression 2	Unifier
King(x)	King(John)	{x/John}
Brother(x, y)	Brother(John, Richard)	{x/John, y/Richard}
Knows(x, Father(x))	Knows(John, y)	{x/John, y/Father(John)}
P(x, x)	P(a, b)	FAIL (cannot unify x with both a and b)

Most General Unifier (MGU)

The MGU is the most general substitution that makes two expressions identical. It contains the minimum number of bindings.

Example

```
Unify(King(x), King(y)) = {x/y} or {y/x}
Both are MGUs - they're equivalent

Unify(P(x), P(f(y))) = {x/f(y)}
```

2.7 Resolution

What is Resolution?

Resolution is a powerful theorem-proving technique that uses **proof by contradiction**. It was invented by John Alan Robinson in 1965.

Principle

1. Negate the statement to be proven
2. Add to knowledge base
3. Apply resolution rules until:
    - **Empty clause (⊥) derived** → Contradiction → Statement is true

- **No more resolutions possible** → Statement cannot be proven

---

Resolution Algorithm

```
def resolution(KB, query):
    """
    Resolution Theorem Proving

    Args:
        KB: Knowledge base (set of clauses)
        query: Statement to prove

    Returns:
        True if query is entailed, False otherwise
    """
    # Negate the query
    negated_query = negate(query)

    # Convert to CNF and add to KB
    clauses = KB.clauses + to_cnf(negated_query)

    # Resolution loop
    new_clauses = set()

    while True:
        pairs = [(c1, c2) for c1 in clauses for c2 in clauses
                 if c1 != c2]

        for ci, cj in pairs:
            # Find resolvable literals
            resolvent = resolve(ci, cj)

            if resolvent is None:
                continue

            if is_empty_clause(resolvent):
                return True  # Contradiction found!

            new_clauses.add(resolvent)

        # Check if any new clauses were added
        if new_clauses.issubset(set(clauses)):
            return False  # No progress

        clauses.extend(new_clauses)
```

Steps for Resolution

1. **Convert to CNF** (Conjunctive Normal Form)

2. **Standardize apart** variables (ensure unique names)
3. **Negate the conclusion**
4. **Apply resolution rule** repeatedly
5. **Derive empty clause** (contradiction) = proof

Converting to CNF

Steps to convert FOL to CNF:

1. **Remove implications**: P → Q becomes ¬P ∨ Q
2. **Move negations inward**: Using De Morgan's laws
3. **Standardize variables**: Rename variables to avoid conflicts
4. **Remove existential quantifiers**: Skolemization
5. **Remove universal quantifiers**: (assumed)
6. **Convert to conjunction of disjunctions**

```python
def to_cnf(formula):
    """
    Convert formula to Conjunctive Normal Form

    Steps:
    1. Eliminate biconditionals
    2. Eliminate implications
    3. Move NOT inwards
    4. Standardize variables
    5. Skolemize (remove ∃)
    6. Drop universal quantifiers
    7. Convert to CNF
    """
    # Step 1: Eliminate ↔
    formula = eliminate_biconditional(formula)

    # Step 2: Eliminate →
    formula = eliminate_implication(formula)

    # Step 3: Move NOT inwards
    formula = move_not_inwards(formula)

    # Step 4: Standardize apart
    formula = standardize_apart(formula)

    # Step 5: Skolemize
    formula = skolemize(formula)

    # Step 6: Drop universal quantifiers
    formula = drop_universal_quantifiers(formula)

    # Step 7: Convert to CNF
    formula = distribute_and_over_or(formula)

    return formula
```

Resolution Rule

For two clauses containing complementary literals:

```
(L1 ∨ L2 ∨ ... ∨ Ln)  and  (M1 ∨ M2 ∨ ... ∨ Mm)
-----------------------------------------------------
SUBST(θ, L1 ∨ L2 ∨ ... ∨ Ln-1 ∨ M1 ∨ ... ∨ Mm-1)
```

Where θ unifies Li with ¬Mj

Example

```
Clause 1: ¬King(x) ∨ ¬Greedy(x) ∨ Evil(x)
Clause 2: King(John)

Resolution:
- Complementary literals: King(John) and ¬King(x)
- Unifier: {x/John}
- Resolvent: ¬Greedy(John) ∨ Evil(John)
```

Resolution Example

Problem: Given KB:

1. ∀x (King(x) ∧ Greedy(x) → Evil(x))
2. King(John)
3. Greedy(John)

Prove: Evil(John)

Solution:

```
Step 1: Convert KB to CNF
1. ¬King(x) ∨ ¬Greedy(x) ∨ Evil(x)
2. King(John)
3. Greedy(John)

Step 2: Negate conclusion
¬Evil(John)

Step 3: Resolution

1. ¬King(x) ∨ ¬Greedy(x) ∨ Evil(x)
2. ¬Evil(John)
   Resolvent: ¬King(John) ∨ ¬Greedy(John)

3. King(John)
4. ¬King(John) ∨ ¬Greedy(John)
   Resolvent: ¬Greedy(John)

5. Greedy(John)
6. ¬Greedy(John)
   Resolvent: Empty clause (⊥)

Empty clause derived → Contradiction!
Therefore: Evil(John) is TRUE
```

Additional Unit 2: Formulas, Solved Examples & Practice Questions

Important Formulas Summary

Propositional Logic Formulas

Formula	Name	Description
$\neg(P \land Q) \equiv (\neg P \lor \neg Q)$	De Morgan's Law 1	Negation of conjunction
$\neg(P \lor Q) \equiv (\neg P \land \neg Q)$	De Morgan's Law 2	Negation of disjunction
$P \to Q \equiv \neg P \lor Q$	Implication Equivalence	Implication to disjunction
$P \leftrightarrow Q \equiv (P \to Q) \land (Q \to P)$	Biconditional	Two-way implication
$P \to Q \equiv \neg Q \to \neg P$	Contrapositive	Equivalent implication
$(P \land Q) \to R \equiv P \to (Q \to R)$	Exportation	Restructures implication

First-Order Logic Formulas

Formula	Name	Description
$\forall x\, P(x) \equiv \neg\exists x\, \neg P(x)$	Universal-Existential	Equivalence of quantifiers
$\exists x\, P(x) \equiv \neg\forall x\, \neg P(x)$	Existential-Universal	Equivalence of quantifiers
$\forall x\, (P(x) \land Q(x)) \equiv (\forall x\, P(x) \land \forall x\, Q(x))$	Universal Distribution	Distribution over $\land$
$\exists x\, (P(x) \lor Q(x)) \equiv (\exists x\, P(x) \lor \exists x\, Q(x))$	Existential Distribution	Distribution over $\lor$
$\neg\forall x\, P(x) \equiv \exists x\, \neg P(x)$	Negation of Universal	Move $\neg$ inside
$\neg\exists x\, P(x) \equiv \forall x\, \neg P(x)$	Negation of Existential	Move $\neg$ inside

Unification Formulas

Concept	Formula	Description
Substitution	$\theta = \{x_1/t_1, x_2/t_2, ..., x_n/t_n\}$	Variable to term mapping
Application	$SUBST(\theta, P(x)) = P(t)$	Apply substitution
Composition	$\theta_1 \circ \theta_2$	Apply θ_1 then θ_2
Most General Unifier	$MGU(p, q)$	Minimal substitution

Resolution Formulas

Formula	Description
$(L_1 \vee ... \vee L_n) \oplus (M_1 \vee ... \vee M_m) = SUBST(\theta, L_1 \vee ... \vee L_{n-1} \vee M_1 \vee ... \vee M_{m-1})$	Resolution rule where θ unifies L_i with $\neg M_j$
$CNF = \wedge_i (\vee_j L_{ij})$	Conjunctive Normal Form
Empty Clause $\perp$	Contradiction indicator

Solved Examples with Step-by-Step Solutions

Example 1: Propositional Logic - Truth Table

Problem: Construct truth table for $(P \rightarrow Q) \wedge (Q \rightarrow P)$

Solution:

P	Q	P→Q	Q→P	(P→Q)∧(Q→P)
T	T	T	T	**T**
T	F	F	T	**F**
F	T	T	F	**F**
F	F	T	T	**T**

Explanation:

- When P=T, Q=T: Both implications true, conjunction true
- When P=T, Q=F: P→Q false, Q→P true, conjunction false
- When P=F, Q=T: P→Q true, Q→P false, conjunction false
- When P=F, Q=F: Both implications true, conjunction true

Example 2: FOL - Translation to Logic

Problem: Translate to FOL: a) "Every student passes some exam" b) "There is a book that every student reads"

Solution:

a) **Every student passes some exam**

```
∀x (Student(x) → ∃y (Exam(y) ∧ Pass(x, y)))
```

b) **There is a book that every student reads**

```
∃y (Book(y) ∧ ∀x (Student(x) → Reads(x, y)))
```

Example 3: Unification - Detailed Steps

Problem: Unify: Knows(x, Father(x)) and Knows(John, y)

Solution:

Step	Action	Result
1	Compare predicates	Both are "Knows" - Match!
2	Compare arguments	Knows(x, Father(x)) vs Knows(John, y)
3	Unify first args	x = John → θ_1 = {x/John}
4	Apply to second	SUBST(θ_1, Father(x)) = Father(John)
5	Unify with y	y = Father(John) → θ_2 = {y/Father(John)}
6	Combine	θ = {x/John, y/Father(John)}

Final Unifier: {x/John, y/Father(John)}

Example 4: Forward Chaining - Step by Step

Problem: Given KB:

- American(West)
- Owns(West, Missy)
- ∀x (American(x) ∧ Dog(y) ∧ Owns(x,y) → Loves(x,y))
- Dog(z)

Prove: ∃x Loves(x, Missy)

Solution:

```
Step 1: Initial facts
Facts: {American(West), Owns(West, Missy), Dog(Missy)}

Step 2: Apply rule ∀x (American(x) ∧ Dog(y) ∧ Owns(x,y) → Loves(x,y))
- Match with x=West, y=Missy
- All premises satisfied!

Step 3: Infer conclusion
Loves(West, Missy)

Step 4: Goal achieved
∃x Loves(x, Missy) is TRUE (x = West)
```

Example 5: Backward Chaining - Step by Step

Problem: KB:

1. ∀x (Grandparent(x, z) ← Parent(x, y) ∧ Parent(y, z))
2. ∀x ∀y (Parent(x, y) ← Mother(x, y))
3. Mother(John, Mary)

Goal: Prove Grandparent(John, Mary)

Solution:

```
Goal: Grandparent(John, Mary)
→ Need: Parent(John, y) ∧ Parent(y, Mary)

Step 1: Try rule 1
- To prove Grandparent(John, Mary)
- Need Parent(John, y) AND Parent(y, Mary)

Step 2: First premise - Parent(John, y)
- Use rule 2: Mother(x,y) → Parent(x,y)
- With x=John, need Mother(John, Mary)
- Found in KB! → Parent(John, Mary) TRUE

Step 3: Second premise - Parent(y, Mary)
- y = Mary
- Need Parent(Mary, Mary) - Cannot prove!

Step 4: Backtrack and try alternative
- Only one possible path

FAILED: Cannot prove Grandparent(John, Mary)
```

Example 6: Resolution - Complete Proof

Problem: KB:

1. ∀x (Teaches(x, y) ∧ Student(y) → Knows(x, y))
2. Teaches(ProfA, Alice)
3. Student(Alice)

Prove: Knows(ProfA, Alice)

Solution:

```
Step 1: Convert to CNF

1. ¬Teaches(x, y) ∨ ¬Student(y) ∨ Knows(x, y)
2. Teaches(ProfA, Alice)
3. Student(Alice)

Step 2: Negate conclusion
¬Knows(ProfA, Alice)

Step 3: Resolution Process

Clause 1: ¬Teaches(x, y) ∨ ¬Student(y) ∨ Knows(x, y)
Clause 2: ¬Knows(ProfA, Alice)
→ Can't resolve directly

Clause 2 + KB2: Resolve Knows(ProfA, Alice) with ¬Knows(x,y)
  with substitution {x/ProfA, y/Alice}
→ Resolvent: ¬Teaches(ProfA, Alice) ∨ ¬Student(Alice)

Clause 3: Student(Alice)
Clause 4: ¬Teaches(ProfA, Alice) ∨ ¬Student(Alice)
→ Resolve with Student(Alice)
→ Resolvent: ¬Teaches(ProfA, Alice)

Clause 2: Teaches(ProfA, Alice)
Clause 5: ¬Teaches(ProfA, Alice)
→ RESOLVENT: EMPTY CLAUSE (⊥)

Step 4: Conclusion
Empty clause derived → Contradiction
∴ Knows(ProfA, Alice) is TRUE ✓
```

Example 7: Skolemization

Problem: Eliminate existential quantifier from ∃x (Person(x) ∧ Loves(x, Mary))

Solution:

```
Original: ∃x (Person(x) ∧ Loves(x, Mary))

Step 1: Identify existential variable
x is existentially quantified

Step 2: Introduce Skolem constant
Replace x with new constant (say, 'a')

Step 3: Result
Person(a) ∧ Loves(a, Mary)

Note: a is a Skolem constant representing "the person who loves Mary"
```

Problem 2: ∃x ∀y Loves(x, y)

```
Original: ∃x ∀y Loves(x, y)

Step 1: x is existential, y is universal
Step 2: Skolemize x with constant: a
Step 3: Since y is universal, replace with Skolem function: f(y)

Result: Loves(a, y)
```

Example 8: Binary Resolution

Problem: Resolve: $C_1 = P(x) \vee Q(a)$ $C_2 = \neg P(b) \vee R(x)$

Solution:

```
Step 1: Identify complementary literals
- C₁ has P(x)
- C₂ has ¬P(b)

Step 2: Check if can unify
- Need to unify P(x) with P(b)
- Unifier: {x/b}

Step 3: Apply unifier and resolve
- Apply {x/b} to C₁: P(b) ∨ Q(a)
- Apply {x/b} to C₂: ¬P(b) ∨ R(b)
- Resolve: Remove P(b) and ¬P(b)

Step 4: Result
Resolvent: Q(a) ∨ R(b)
```

Additional Practice Questions with Solutions

Question 1: Propositional Logic

Q: Show that $(P \rightarrow Q)$ is logically equivalent to $(\neg P \vee Q)$

Solution:

```
Using truth table:

P → Q = ¬P ∨ Q

P Q | P→Q | ¬P | ¬P∨Q
T T |  T  |  F |   T
T F |  F  |  F |   F
F T |  T  |  T |   T
F F |  T  |  T |   T

Both columns are identical → Logically Equivalent ✓
```

Question 2: FOL Translation

Q: Translate: "No one likes everyone"

Solution:

```
Method 1: Using ¬∃
¬∃x (Person(x) ∧ ∀y (Person(y) → Likes(x, y)))

Method 2: Using ∀
∀x (Person(x) → ¬∀y (Person(y) → Likes(x, y)))

Simplified (Method 3):
∀x ∀y (Person(x) ∧ Person(y) → ¬Likes(x, y))

All three are logically equivalent!
```

Question 3: Unification

Q: Can we unify P(f(x), g(y)) and P(a, z)? If yes, find MGU.

Solution:

```
Step 1: Check predicate - Both P ✓

Step 2: Compare first arguments
- f(x) vs a
- Cannot unify function with constant directly
- Need x = f⁻¹(a) - NOT POSSIBLE!

Answer: UNIFICATION FAILS

Reason: Cannot map function f(x) to constant a
         without knowing what f represents
```

Question 4: Forward Chaining Trace

Q: Given KB:

- Fast(BMW)
- Car(x) → Vehicle(x)
- BMW(x) → Car(x)
- Vehicle(x) ∧ Fast(x) → SportsCar(x)

Show forward chaining proves SportsCar(BMW)

Solution:

```
Iteration 1:
- Add Fast(BMW) to inferred facts

Iteration 2: Apply rule BMW(x) → Car(x)
- Match: x = BMW
- Conclusion: Car(BMW)
- New facts: {Car(BMW)}

Iteration 3: Apply rule Car(x) → Vehicle(x)
- Match: x = BMW
- Conclusion: Vehicle(BMW)
- New facts: {Vehicle(BMW)}

Iteration 4: Apply rule Vehicle(x) ∧ Fast(x) → SportsCar(x)
- Match: x = BMW (both Vehicle(BMW) and Fast(BMW) true!)
- Conclusion: SportsCar(BMW)

Iteration 5: Check goal - SportsCar(BMW) PROVED ✓
```

Question 5: Resolution Proof

Q: Prove modus tollens using resolution: Given: P → Q, ¬Q Prove: ¬P

Solution:

```
Step 1: Convert to CNF
- P → Q becomes ¬P ∨ Q
- ¬Q stays ¬Q
- ¬P stays ¬P (to prove)

Step 2: Add negated conclusion to KB
KB: {¬P ∨ Q, ¬Q, P} (adding ¬(¬P) = P)

Step 3: Resolution

Clause 1: ¬P ∨ Q
Clause 2: ¬Q
→ Resolve Q and ¬Q: {¬P}

Clause 3: P (from negated conclusion)
Clause 4: ¬P
→ Resolve P and ¬P: EMPTY CLAUSE ⊥

Step 4: Conclusion
Empty clause derived → Contradiction
∴ ¬P is proved ✓
```

Question 6: Occurs Check

Q: Does unification succeed for P(x, f(x)) and P(f(y), y)?

Solution:

```
Step 1: Try unify x with f(y)
- x = f(y)
- Apply to second argument: f(f(y))

Step 2: Now need to unify f(y) with y
- Check occurs check: Does y appear in f(y)? YES!

Step 3: Occurs check FAILS
- Cannot unify variable with expression containing that variable
- x = f(y) would lead to infinite term f(f(f(...)))

Answer: UNIFICATION FAILS (Occurs Check)
```

More Exam-Oriented Questions

Short Answer Questions (2-3 marks each)

Q1: Define an atom in FOL.

```
Answer: An atom (atomic sentence) is a predicate applied to terms,
        e.g., Brother(John, King), Likes(x, y). It has no quantifiers
        or logical connectives within it.
```

Q2: What is the difference between forward and backward chaining?

```
Answer: Forward chaining is data-driven (bottom-up), starts from
        known facts and derives conclusions. Backward chaining is
        goal-driven (top-down), starts from goal and works backwards
        to find supporting facts.
```

Q3: What is a Skolem constant?

```
Answer: A Skolem constant replaces an existentially quantified variable
        during Skolemization. It represents "some object" without
        introducing new constraints.
```

Q4: Define Modus Ponens in propositional logic.

```
Answer: From P → Q and P, infer Q. If premise is true and implication
        is true, conclusion must be true.
```

Q5: What is an admissible heuristic?

```
Answer: A heuristic h(n) is admissible if it never overestimates
        the cost to reach the goal. It must be optimistic (never
        overestimate actual cost).
```

Long Answer Questions (10-15 marks each)

Q1: Explain the resolution theorem proving process with a complete example.

Answer:

```
Resolution is a complete inference method for first-order logic.

Steps:
1. Convert all KB sentences to CNF
2. Negate the conclusion
3. Add negated conclusion to KB
4. Repeatedly resolve pairs of clauses
5. If empty clause derived → proof complete
6. If no progress → proof failed

Example Proof:
Given: ∀x (King(x) ∧ Greedy(x) → Evil(x))
       King(John)
       Greedy(John)
Prove: Evil(John)

(Complete solution shown in notes above - see Example 6)
```

Q2: Compare forward chaining and backward chaining with advantages and disadvantages.

Answer:

```
Forward Chaining:
- Data-driven, bottom-up
- Starts from known facts
- Derives all possible conclusions
- Used in expert systems, production systems
- Advantages: Derives hidden knowledge
- Disadvantages: May derive irrelevant facts

Backward Chaining:
- Goal-driven, top-down
- Starts from goal to prove
- Only explores relevant paths
- Used in Prolog, diagnostic systems
- Advantages: Focused, efficient
- Disadvantages: May miss useful facts
```

Q3: Explain unification algorithm with example. What is occurs check?

Answer:

```
Unification finds substitution making two expressions identical.

Algorithm:
1. If expressions identical, return empty substitution
2. If either is variable, apply occurs check, return binding
3. If both compound, check same predicate, unify args recursively
4. Otherwise, fail

Occurs Check:
- Prevents unifying x with expression containing x
- Example: P(x) cannot unify with P(f(x)) because x appears in f(x)
- Prevents infinite terms like f(f(f(...)))

Without occurs check: Would create infinite-length terms
```

Unit Summary & Practice Questions

Unit 1: Quick Reference Card

Concept	Algorithm	Key Formula	Example
BFS	Breadth-First	O(b^d)	Level-by-level
DFS	Depth-First	O(bm)	Deep-first
UCS	Uniform Cost	O(b^(1+C*/ε))	Minimum cost
IDDFS	Iterative Deepening	O(b^d)	BFS+DFS
Greedy	Best-First	f(n)=h(n)	Closest to goal
A*	A-Star	f(n)=g(n)+h(n)	Optimal+fast

Unit 2: Quick Reference Card

Concept	Description	Use Case
Propositional Logic	True/false statements	Simple reasoning
First-Order Logic	Quantifiers + Predicates	Complex relationships
Unification	Find common substitution	Matching patterns
Forward Chaining	Facts → Goal	Data-driven
Backward Chaining	Goal → Facts	Goal-driven
Resolution	Proof by contradiction	Theorem proving

Key Formulas to Remember

Topic	Formula
Implication	$P \rightarrow Q \equiv \neg P \vee Q$
Biconditional	$P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$
De Morgan 1	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
De Morgan 2	$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
Universal-Existential	$\forall x\ P(x) \equiv \neg\exists x\ \neg P(x)$
*A Evaluation**	$f(n) = g(n) + h(n)$
Resolution	$(L \vee A) \oplus (\neg L \vee B) = A \vee B$
Admissible Heuristic	$h(n) \leq h^*(n)$ (never overestimate)

Practice Questions

Unit 1: Problem Solving

Easy

1. Define an intelligent agent and explain its components.
2. What is the difference between BFS and DFS?
3. What is a heuristic function?
4. Define PEAS representation.

Medium

1. Explain the working of A* search algorithm with an example.
2. Compare Uninformed and Informed search strategies.
3. What is the difference between hill climbing and A* search?
4. Explain constraint satisfaction with an example.

Hard

1. Show step-by-step execution of A* search for a given graph.
2. Compare all uninformed search algorithms in terms of completeness and optimality.
3. Explain how IDDFS combines benefits of BFS and DFS.

Unit 2: Logical Reasoning

Easy

1. Define propositional logic and give its limitations.
2. What is the difference between ∀ and ∃ quantifiers?
3. Explain unification with an example.
4. What is forward chaining?

Solutions - Easy:

```
Q1: Propositional logic deals with true/false propositions using connectives.
    Limitations: Cannot express relationships, no quantifiers, must enumerate
    all facts.

Q2: ∀ (Universal): "For all" - property holds for ALL objects
    ∃ (Existential): "There exists" - property holds for AT LEAST ONE object

Q3: Unification finds substitution making two expressions identical.
    Example: Unify King(x) and King(John) → {x/John}

Q4: Forward chaining starts from facts and applies rules to derive conclusions.
    Data-driven, bottom-up approach.
```

Medium

1. Convert "All students who study pass" to FOL and prove using resolution.
2. Explain the difference between forward and backward chaining.
3. What is the resolution principle? Explain with an example.
4. Convert the following to CNF: ∀x (P(x) → Q(x))

Solutions - Medium:

```
Q1: FOL: ∀x (Student(x) ∧ Studies(x) → Pass(x))

Q2: Forward Chaining: Facts → Goal (data-driven)
    Backward Chaining: Goal → Facts (goal-driven)

Q3: Resolution uses proof by contradiction. Negate conclusion, add to KB,
    resolve until empty clause derived.

Q4: CNF Conversion:
    ∀x (P(x) → Q(x))
    = ∀x (¬P(x) ∨ Q(x))
    = ¬P(x) ∨ Q(x)  [drop universal]
```

Hard

1. Prove using resolution: Given KB: ∀x (King(x) ∧ Greedy(x) → Evil(x)), King(John), Greedy(John), Prove: Evil(John)
2. Explain the unification algorithm with a complex example.
3. Compare propositional logic and first-order logic.

Solutions - Hard:

```
Q1: Resolution Proof:
    1. KB in CNF: ¬King(x) ∨ ¬Greedy(x) ∨ Evil(x), King(John), Greedy(John)
    2. Negate conclusion: ¬Evil(John)
    3. Resolve:
       - ¬Evil(John) + (¬King(x) ∨ ¬Greedy(x) ∨ Evil(x)) with {x/John}
         → ¬King(John) ∨ ¬Greedy(John)
       - King(John) + ¬King(John) ∨ ¬Greedy(John)
         → ¬Greedy(John)
       - Greedy(John) + ¬Greedy(John) → EMPTY CLAUSE ⊥
    4. PROVED! ✓

Q2: [See detailed explanation in notes above - Unification Section]

Q3: [See comparison table in notes]
```

Common Errors & Solutions

Error	Cause	Solution
A* not optimal	Non-admissible heuristic	Use admissible h(n)
Infinite loop in DFS	No cycle checking	Maintain visited set
BFS memory overflow	Large branching factor	Use iterative deepening
Unification failure	Occurs check fails	Use occurs check
Resolution doesn't terminate	Infinite search	Set maximum iterations

---

Top Exam Topics (Must Revise)

Unit 1

1. **Intelligent Agents & PEAS** - 5-10 marks
2. **BFS vs DFS** - 10 marks
3. *A Search Algorithm** - 15 marks (most important!)
4. **Hill Climbing** - 5-8 marks
5. **Constraint Satisfaction** - 8-10 marks

Unit 2

1. **Propositional vs First-Order Logic** - 10 marks
2. **Unification** - 10 marks
3. **Forward vs Backward Chaining** - 10 marks
4. **Resolution** - 15 marks (most important!)
5. **Quantifiers** - 5-8 marks

Previous Year Examination Questions & Detailed Answers

QUESTION PAPER - 1

Q1 (a) What is a Knowledge Based System? (3 marks)

Answer:

A **Knowledge-Based System (KBS)** is an AI system that uses knowledge representation and reasoning techniques to solve complex problems. It consists of:

Component	Description
Knowledge Base (KB)	Stores facts, rules, and heuristics about the domain
Inference Engine	Derives new conclusions from KB using inference rules
User Interface	Interacts with users to query the system
Explanation Module	Provides reasoning behind conclusions

Example: Medical diagnosis system, Expert systems

```
┌──────────────────────────────────────┐
│         Knowledge-Based              │
│              System                  │
├──────────────────────────────────────┤
│  ┌──────────────┐  ┌────────────────┐ │
│  │  Knowledge   │  │   Inference    │ │
│  │    Base      │  │    Engine      │ │
│  └──────────────┘  └────────────────┘ │
└──────────────────────────────────────┘
```

Q1 (b) Give the basic algorithm of depth first search. (3 marks)

Answer:

Depth First Search (DFS) explores as deep as possible before backtracking.

```python
def dfs(graph, start, goal, visited=None, path=None):
    """DFS Algorithm"""
    if visited is None:
        visited = set()
    if path is None:
        path = []

    path = path + [start]
    visited.add(start)

    if start == goal:
        return path

    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            result = dfs(graph, neighbor, goal, visited, path)
            if result:
                return result

    return None
```

Algorithm Steps:

1. Start with root node, push to stack
2. Pop node from stack, mark as visited
3. If node is goal, return solution
4. Push all unvisited neighbors to stack
5. Repeat until stack empty or goal found

Q1 (c) What is propositional logic? How it is useful in machine learning? (4 marks)

Answer:

Propositional Logic is a formal system dealing with propositions (true/false statements) and logical connectives.

Components:

- **Propositions:** P, Q, R (True/False)
- **Connectives:** ∧ (AND), ∨ (OR), ¬ (NOT), → (IMPLIES), ↔ (IFF)

Usefulness in Machine Learning:

Application	How Propositional Logic Helps
Decision Trees	Each node represents a propositional condition
Rule-Based ML	IF-THEN rules use propositional logic
Neural Network Verification	Formal properties expressed in propositional logic
Feature Selection	Boolean feature combinations
Ensemble Methods	Voting logic uses AND/OR operations

Example:

```
IF Temperature > 30 AND Humidity < 50 THEN Hot_Day = True
```

Q2 (a) Compare Informed & Uninformed search with examples (5 marks)

Answer:

Aspect	Uninformed Search	Informed Search
Other Name	Blind Search	Heuristic Search
Knowledge	No domain knowledge	Uses heuristic function
Examples	BFS, DFS, UCS	A*, Greedy Best-First
Efficiency	Less efficient	More efficient
Optimal	Sometimes	Always (with good heuristic)

Uninformed Search Examples:

- **BFS:** Explores all nodes at level d before d+1
- **DFS:** Goes deep before backtracking
- **UCS:** Expands lowest cost path

Informed Search Examples:

- **Greedy Best-First:** Expands node closest to goal (h(n))
- *A Search:** Expands node with lowest f(n) = g(n) + h(n)

Comparison with Water Jug Problem:

```
Uninformed: Try all possible combinations
Informed: Use heuristic like "minimize water transferred"
```

Q2 (b) Analyze the logic behind breadth first search algorithm (5 marks)

Answer:

Breadth First Search (BFS) explores all nodes at current level before moving to next level.

Algorithm:

```python
from collections import deque

def bfs(graph, start, goal):
    queue = deque([(start, [start])])
    visited = set()

    while queue:
        node, path = queue.popleft()

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)
            for neighbor in graph.get(node, []):
                if neighbor not in visited:
                    queue.append((neighbor, path + [neighbor]))
    return None
```

Logic Analysis:

Property	Value
Data Structure	Queue (FIFO)
Completeness	Yes, finds solution if exists
Optimality	Yes, finds shortest path
Time Complexity	O(b^d)
Space Complexity	O(b^d)

Why BFS works:

- Uses FIFO to explore level by level
- Guarantees shortest path for unweighted graphs
- Suitable when goal is close to start

Q3 (a) Discuss Resolution in brief with an example (5 marks)

Answer:

Resolution is a theorem-proving technique using proof by contradiction.

Steps:

1. Convert all KB sentences to CNF (Conjunctive Normal Form)
2. Negate the conclusion
3. Apply resolution rule repeatedly
4. If empty clause (⊥) derived → Proved!

Resolution Rule: ``` (L1 ∨ L2 ∨ ... ∨ Ln) and (¬L1 ∨ M1 ∨ ... ∨ Mm)

SUBST(θ, L2 ∨ ... ∨ Ln ∨ M1 ∨ ... ∨ Mm)

```
**Example:**

Given:
1. ∀x (King(x) ∧ Greedy(x) → Evil(x))
2. King(John)
3. Greedy(John)

Prove: Evil(John)
```

Step 1: CNF

- ¬King(x) ∨ ¬Greedy(x) ∨ Evil(x)
- King(John)
- Greedy(John)

Step 2: Negate conclusion: ¬Evil(John)

Step 3: Resolution

- ¬Evil(John) + [¬King(x) ∨ ¬Greedy(x) ∨ Evil(x)] with {x/John} → ¬King(John) ∨ ¬Greedy(John)
- King(John) + ¬King(John) ∨ ¬Greedy(John) → ¬Greedy(John)
- Greedy(John) + ¬Greedy(John) → EMPTY CLAUSE ⊥

✓ PROVED!

```
---

### Q3 (b) Illustrate forward and backward chaining with suitable example (5 marks)

**Answer:**

**Forward Chaining (Data-Driven):**
- Starts from known facts
- Applies rules to derive conclusions
- Used in **deductive databases**

**Example:**
```

KB:

1. King(John)
2. Greedy(John)
3. ∀x (King(x) ∧ Greedy(x) → Evil(x))

Forward Chaining: Step 1: Facts: {King(John), Greedy(John)} Step 2: Apply rule 3 with x=John Step 3: Infer: Evil(John)

```
**Backward Chaining (Goal-Driven):**
- Starts from goal
- Works backwards to find supporting facts
- Used in **Prolog**, **Expert Systems**

**Example:**
```

Goal: Evil(John)

Step 1: Find rule with Evil(x) ∀x (King(x) ∧ Greedy(x) → Evil(x))

Step 2: Need to prove King(John) ∧ Greedy(John) - King(John) is a fact ✓ - Greedy(John) is a fact ✓

Step 3: Goal PROVED ✓

```
| Aspect | Forward Chaining | Backward Chaining |
|--------|------------------|-------------------|
| Direction | Facts → Goal | Goal → Facts |
| Method | Bottom-up | Top-down |
| Control | Data-driven | Goal-driven |
| Use | Deductive DBs | Expert Systems |

---

### Q4 (a) What do you mean by iterative deepening? (5 marks)

**Answer:**

**Iterative Deepening Depth-First Search (IDDFS)** combines BFS and DFS benefits by gradually
increasing depth limit.

**How it works:**
```

Iteration 1: Depth limit = 0 → Visit only root Iteration 2: Depth limit = 1 → Visit root + children Iteration 3: Depth limit = 2 → Visit root + 2 levels ... Continue until goal found

```
**Algorithm:**
```python
def iterative_deepening_dfs(graph, start, goal, max_depth=100):
    for limit in range(max_depth):
        result = depth_limited_search(graph, start, goal, limit)
        if result and result != 'cutoff':
            return result
    return None
```
```

Advantages:

Advantage	Description
Memory Efficient	Like DFS, stores only one path
Optimal	Like BFS, finds shortest path
Complete	Will find solution if exists
Combines BFS+DFS	Best of both worlds

Time Complexity: O(b^d) **Space Complexity:** O(bd)

Q4 (b) How to define a problem as state space search? Discuss with

example (5 marks)

Answer:

State Space Search represents problem as graph of states with operators/transitions.

Components:

Component	Description
Initial State	Starting state
Goal State	Target state(s)
Operators/Actions	Valid moves between states
State Space	All possible states
Path Cost	Cost of traversing

Example: Water Jug Problem

```
Problem: Given 4L and 3L jugs, get exactly 2L in 4L jug

Initial State: (0, 0)  [both jugs empty]
Goal State:   (2, n)   [2L in 4L jug]

Operators:
1. Fill 4L jug: (x, y) → (4, y)
2. Fill 3L jug: (x, y) → (x, 3)
3. Empty 4L jug: (x, y) → (0, y)
4. Empty 3L jug: (x, y) → (x, 0)
5. Pour 4L→3L: (x, y) → (x-3, y+3) if x+y≥3
6. Pour 3L→4L: (x, y) → (x+3, y-3) if x+y≥4

State Space Graph:
(0,0) → (4,0) → (4,3) → (1,3) → (1,0) → ... → (2,0)
```

QUESTION PAPER - 2

Q1 (a) AI is interdisciplinary in nature and its foundations are in various fields. Justify the statement. (2 marks)

Answer:

AI draws from multiple disciplines:

Field	Contribution to AI
Mathematics	Logic, probability, statistics, algorithms
Philosophy	Mind, consciousness, ethics
Psychology	Cognitive processes, learning
Linguistics	Natural language processing
Computer Science	Algorithms, programming, hardware
Neuroscience	Neural networks, brain modeling
Economics	Decision theory, game theory
Biology	Evolutionary algorithms, genetic algorithms

Q1 (b) What are the problems associated with propositional logic? (2 marks)

Answer:

Problem	Description
Limited Expressiveness	Cannot represent relationships between objects
No Quantifiers	Cannot express "all" or "some"
No Functions	Cannot map objects to other objects
Propositional Explosion	Must enumerate all facts
No Common Sense	Cannot reason about everyday knowledge

Example Problem:

```
Cannot express: "All students pass some exam"
Must write: Student1→Pass1 ∧ Student2→Pass2 ∧ ... (infinite!)
```

Q1 (c) Explain the characteristics and limitations of uninformed search algorithms. (2 marks)

Answer:

Characteristics:

- Also called **Blind Search**
- No domain knowledge used
- Only knows how to traverse tree
- Examines every node until goal found

Limitations:

Limitation	Impact
Exponential Time	O(b^d) - very slow
High Memory	Stores all nodes in memory
Blind	Cannot prioritize promising paths
Not Optimal	Some (DFS) don't find shortest path

Q1 (d) Is heuristic search always better than blind search? Justify your answer. (2 marks)

Answer:

NO, heuristic search is NOT always better.

When Heuristic Search is Better:

- Domain-specific knowledge available
- Good admissible heuristic exists

- Search space is large

When Blind Search is Better:

- No heuristic available
- Solution space is small
- Need guaranteed optimal solution

Justification:

```
A* with poor heuristic → May perform worse than BFS
Heuristic with high computation cost → Overhead may not be worth it
Blind search → Predictable time/memory bounds
```

Q1 (e) Differentiate between forward and backward chaining. (2 marks)

Forward Chaining	Backward Chaining
Facts → Goal	Goal → Facts
Bottom-up	Top-down
Data-driven	Goal-driven
Derives all conclusions	Proves specific goal
Forward: B→C, C→D, D→Goal	Backward: Goal←B, B←C, C←A

Q2 (a) What do you mean by Artificial Intelligence? How you define problem as a state space search? Explain using water-jug problem. (5 marks)

Answer:

Artificial Intelligence is the study of intelligent agents that perceive, reason, and act to achieve goals.

State Space Search for Water Jug Problem:

```
Problem: Measure exactly 2L using 4L and 3L jugs

State Representation: (x, y) where x = water in 4L jug, y = water in 3L jug

Initial State: (0, 0)

Goal State: (2, any)

Operators:
1. Fill 4L: (x,y) → (4,y)
2. Fill 3L: (x,y) → (x,3)
3. Empty 4L: (x,y) → (0,y)
4. Empty 3L: (x,y) → (x,0)
5. Pour 4L→3L: (x,y) → max(0, x-(3-y)), min(3, x+y)
6. Pour 3L→4L: (x,y) → min(4, x+y), max(0, y-(4-x))

Solution Path:
(0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3) ✓
```

Q2 (b) Explain how heuristic function helps in the process of searching? Discuss Best First Search algorithm with example. (5 marks)

Answer:

Heuristic Function h(n):

- Estimates cost from node n to goal
- Guides search toward promising nodes
- Makes search efficient

Best First Search:

- Expands node with lowest h(n)
- Combines DFS and BFS benefits
- Formula: f(n) = h(n)

Example:

```
Graph:
        S(0)
       / \
   A(1)/   \B(2)
     /      \
   C(4)     F(5)
    |\      /|
   D(5)   G(6)
    |     /|
    K(6)--+

Heuristic h(n): S=6, A=4, B=5, C=3, D=4, F=4, G=3, K=0

Step-by-step:
1. OPEN: {S(6)} → CLOSED: {}
2. Expand S: {A(4), B(7)} → CLOSED: {S}
3. Expand A: {C(4), B(7), D(8)} → CLOSED: {S, A}
4. Expand C: {D(6), G(6), B(7)} → CLOSED: {S, A, C}
5. Expand G: {K(6)} → GOAL!

Path: S → A → C → G → K
```

Q3 (a) Explain constraint satisfaction problem with the help of an example. (5 marks)

Answer:

Constraint Satisfaction Problem (CSP):

A CSP has:

- Set of variables $\{X_1, X_2, ..., X_n\}$
- Each variable has domain D_i
- Set of constraints C

Example: Cryptarithmetic

```
  SEND
+ MORE
------
 MONEY

Variables: S,E,N,D,M,O,R,Y (each = 0-9)
Constraints:
1. All digits unique
2. Leading digits ≠ 0
3. Column arithmetic satisfied

Solution:
  9567
+ 1085
------
 10652

S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2
```

Solving Methods:

- Backtracking
- Forward Checking
- Constraint Propagation

Q3 (b) How is unification used in resolution? Explain with example. (5 marks)

Answer:

Unification finds substitution making two expressions identical.

In Resolution:

- Finds complementary literals
- Applies substitution to resolve clauses

Example:

```
Clause 1: ¬King(x) ∨ Evil(x)
Clause 2: King(John)

Step 1: Find complementary literals
        ¬King(x) and King(John)

Step 2: Unify with θ = {x/John}

Step 3: Apply substitution
        Clause 1: ¬King(John) ∨ Evil(John)
        Clause 2: King(John)

Step 4: Resolve
        Result: Evil(John)
```

Unification Algorithm:

```
# Key steps:
1. If x == y: return θ
2. If is_variable(x): unify_var(x, y, θ)
3. If is_compound(x) and y: unify args recursively
4. If cannot unify: FAIL
```

Q4 (a) Differentiate between breadth first search and depth first search with the help of example. (5 marks)

Answer:

Aspect	BFS	DFS
Data Structure	Queue	Stack
Strategy	Level by level	Deep first
Optimal	Yes	No
Complete	Yes	No (can loop)
Memory	High	Low
Time	O(b^d)	O(b^m)

Example Tree:

```
        S
       / \
      A   B
     /|   |\
    C D   E F

BFS Order: S → A → B → C → D → E → F
DFS Order: S → A → C → D → B → E → F
```

Q4 (b) What do you mean by intelligent agents? Explain various types of agents. (5 marks)

Answer:

Intelligent Agent is an autonomous entity that perceives environment and acts to achieve goals.

PEAS Representation:

Component	Description
Performance	Measure of success
Environment	Where agent operates
Actuators	Actions agent can take
Sensors	How agent perceives

Types of Agents:

Agent Type	Description	Example
Simple Reflex	Responds to current perception	Thermostat
Model-Based	Maintains internal world model	Self-driving car
Goal-Based	Has goals to achieve	Chess player
Utility-Based	Maximizes utility function	Robot planning
Learning Agent	Improves from experience	AlphaGo

End of Document

Notes prepared according to USICT B.Tech AIML Syllabus (AI-316T) Including Previous Year Exam Questions & Detailed Answers